Multi-Source Web Scraping Project

Complete source code - Realisieren Technologies assignment

What this is

A Python ETL pipeline that scrapes Books to Scrape and Quotes to Scrape by following the 'next' links, cleans and validates every record, removes duplicates (even with different spacing or capitalisation) and writes one consolidated CSV, a JSON summary report and a log file. One command runs everything: `python main.py`.

Quick start in VS Code

1. Install Python 3.10, 3.11 or 3.12 and VS Code (with the Python extension).

2. Create a folder named `scraping_assignment` and open it in VS Code (File > Open Folder).

3. Create every file in this PDF at the exact path shown in its blue heading (folders: scrapers, processing, tests) and paste the code. Keep indentation exactly as shown - Python depends on it. The output and logs folders are created automatically.

4. Open the VS Code terminal and run the commands below.

```
python -m venv .venv
# Windows:      .venv\Scripts\activate
# macOS/Linux:  source .venv/bin/activate
pip install -r requirements.txt

python -m unittest discover -s tests -v   # 31 offline tests, all should pass
python main.py --max-pages 2              # quick trial run
python main.py                            # full run (about 2-4 minutes)
```

Results appear in `output/final_dataset.csv`, `output/summary_report.json`, `output/excluded_records.json` and `logs/`.

Project structure

```
scraping_assignment/
  main.py                 command-line entry point
  pipeline.py             scrape -> clean -> validate -> dedupe -> save
  config.py               sources, schema, rules, HTTP settings
  requirements.txt  conftest.py  README.md  AI_USAGE.md
  scrapers/               http_client, base_scraper (pagination),
                          books_scraper, quotes_scraper
  processing/             cleaning, validation, deduplication
  tests/                  helpers + 5 unittest files (no internet needed)
  output/  logs/          created automatically
```

Before you submit (important)

1) Do a full live run on your own machine and check the CSV and JSON. 2) Fill in the two placeholders in AI_USAGE.md with what you really did and found. 3) Add your real output files and a sample log to the ZIP. 4) Read every module - the interview asks you to explain the code, including which parts were AI-assisted.

Note: this code was tested offline (unit tests, a local mock server and the real HTML structure of the Books to Scrape home page). A complete live run against both sites could not be done where it was written, so please run it once yourself.

FILE 1 of 22: requirements.txt

```
# Runtime dependencies (Python 3.10 - 3.12)
# Developed/tested with: requests 2.33.1, beautifulsoup4 4.14.3, lxml 6.0.2
# To freeze the exact versions of YOUR environment run: pip freeze > requirements.lock
requests>=2.31,<3
beautifulsoup4>=4.12,<5
lxml>=5.0,<7

# Tests use the standard-library unittest module, so no extra package is needed.
# (pytest also works if you prefer it: pip install pytest)
```

FILE 2 of 22: config.py

```
"""Central configuration: source names, output schema, rules and defaults."""
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent
DEFAULT_OUTPUT_DIR = PROJECT_ROOT / "output"
DEFAULT_LOG_DIR = PROJECT_ROOT / "logs"

# ---- Sources ---------------------------------------------------------------
BOOKS_SOURCE = "Books to Scrape"
QUOTES_SOURCE = "Quotes to Scrape"
BOOKS_URL = "https://books.toscrape.com/"
QUOTES_URL = "https://quotes.toscrape.com/"
KNOWN_SOURCES = {BOOKS_SOURCE, QUOTES_SOURCE}

# ---- Common output schema (column order of the CSV) ----------------------
SCHEMA = [
    "source",
    "name_or_title",   # book title OR quote text
    "category",        # books only
    "price",           # books only (GBP, float)
    "rating",          # books only (1-5, int)
    "author",          # quotes only
    "tags",            # quotes only (joined with TAG_SEPARATOR in the CSV)
    "availability",    # books only
    "description",     # books only, and only with --with-details
    "source_url",
    "scraped_at",
]
TAG_SEPARATOR = "|"

# ---- Validation rules ------------------------------------------------------
REQUIRED_FIELDS = {
    BOOKS_SOURCE: ("name_or_title", "price", "rating", "source_url", "scraped_at"),
    QUOTES_SOURCE: ("name_or_title", "author", "source_url", "scraped_at"),
}
RATING_MIN, RATING_MAX = 1, 5

# ---- Duplicate detection: fields that identify "the same record" ---------
DEDUP_KEYS = {
    BOOKS_SOURCE: ("name_or_title", "price"),
    QUOTES_SOURCE: ("name_or_title", "author"),
}

# ---- HTTP behaviour --------------------------------------------------------
USER_AGENT = "assignment-scraper/1.0 (educational project)"
DEFAULT_DELAY = 0.5       # seconds between requests
TIMEOUT = 15              # seconds per request
MAX_RETRIES = 3           # attempts per URL
BACKOFF = 1.0             # 1s, 2s, 4s ... between retries
MAX_CONSECUTIVE_PAGE_FAILURES = 2
```

FILE 3 of 22: scrapers/__init__.py

```
"""Source-specific scrapers and the shared HTTP client."""
```

FILE 4 of 22: scrapers/http_client.py

```
"""HTTP layer: rate limiting, timeouts, retries with exponential backoff."""
import logging
import time

import requests
from bs4 import BeautifulSoup

import config

log = logging.getLogger(__name__)

RETRYABLE_STATUS = {429, 500, 502, 503, 504}


class HttpClient:
    """Polite GET client. Never raises for network problems: returns None."""

    def __init__(
        self,
        delay=config.DEFAULT_DELAY,
        timeout=config.TIMEOUT,
        max_retries=config.MAX_RETRIES,
        backoff=config.BACKOFF,
    ):
        self.delay = delay
        self.timeout = timeout
        self.max_retries = max_retries
        self.backoff = backoff
        self.request_count = 0
        self._last_request_at = 0.0
        self.session = requests.Session()
        self.session.headers.update({"User-Agent": config.USER_AGENT})

    def _respect_rate_limit(self):
        wait = self.delay - (time.monotonic() - self._last_request_at)
        if wait > 0:
            time.sleep(wait)

    def get_html(self, url):
        """Return the page HTML as text, or None after all retries failed."""
        for attempt in range(1, self.max_retries + 1):
            self._respect_rate_limit()
            reason = None
            try:
                response = self.session.get(url, timeout=self.timeout)
            except (requests.Timeout, requests.ConnectionError) as exc:
                response, reason = None, type(exc).__name__
            except requests.RequestException as exc:  # e.g. invalid URL
                log.error("Request to %s failed permanently: %s", url, exc)
                return None
            self._last_request_at = time.monotonic()
            self.request_count += 1

            if response is not None:
                if response.status_code == 200:
                    response.encoding = "utf-8"  # sites omit charset; avoid mojibake
                    log.info("GET %s -> 200", url)
                    return response.text
                if response.status_code not in RETRYABLE_STATUS:
                    log.error("GET %s -> HTTP %s (not retried)", url, response.status_code)
                    return None
                reason = f"HTTP {response.status_code}"

            log.warning(
                "GET %s failed (%s) - attempt %d/%d",
                url, reason, attempt, self.max_retries,
            )
            if attempt < self.max_retries:
                time.sleep(self.backoff * 2 ** (attempt - 1))

        log.error("Giving up on %s after %d attempts", url, self.max_retries)
        return None
    def get_soup(self, url):
```

```
        """Return a parsed BeautifulSoup tree, or None when the fetch failed."""
        html = self.get_html(url)
        if html is None:
            return None
        return BeautifulSoup(html, "lxml")
```

FILE 5 of 22: scrapers/base_scraper.py

```
"""Shared scraper machinery: pagination loop and safe extraction helpers."""
import logging
from dataclasses import dataclass, field
from datetime import datetime, timezone
from urllib.parse import urljoin

import config

log = logging.getLogger(__name__)


def utc_now_iso():
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def text_of(element):
    """Text of a BeautifulSoup element, or None when the element is missing."""
    return element.get_text() if element is not None else None


def attr_of(element, name):
    """String attribute of an element, or None when missing."""
    if element is None:
        return None
    value = element.get(name)
    return value if isinstance(value, str) else None


def blank_record(source):
    """A record with every schema column present (None = not applicable)."""
    record = {name: None for name in config.SCHEMA}
    record["source"] = source
    return record


@dataclass
class ScrapeResult:
    source: str
    records: list = field(default_factory=list)
    pages_fetched: int = 0
    pages_failed: int = 0
    failed_urls: list = field(default_factory=list)
    parse_errors: int = 0
    extra_pages_fetched: int = 0   # supporting pages (categories / details)
    extra_pages_failed: int = 0
    error: str = None


class BaseScraper:
    """Pagination loop shared by all sources. Subclasses add selectors."""

    name = ""
    start_url = ""

    def __init__(self, client, start_url=None, max_pages=None):
        self.client = client
        self.start_url = start_url or self.start_url
        self.max_pages = max_pages

    # ---- to be provided by subclasses ----------------------------------
    def parse_page(self, soup, page_url):
        """Return (records, number_of_items_that_failed_to_parse)."""
        raise NotImplementedError

    def guess_next_url(self, page_url):
        """Fallback used only when a page failed and its 'next' link is unknown."""
        return None
```

```
    # ---- shared behaviour -----------------------------------------------
    def find_next_url(self, soup, page_url):
        href = attr_of(soup.select_one("li.next a"), "href")
        return urljoin(page_url, href) if href else None

    def crawl(self, start_url, on_page, stats):
        """Follow 'next' links from start_url. Calls on_page(soup, url) per page."""
        url, visited, failures, attempted = start_url, set(), 0, 0
        while url and url not in visited:
            if self.max_pages is not None and attempted >= self.max_pages:
                log.info("Reached max_pages=%d, stopping this crawl", self.max_pages)
                break
            visited.add(url)
            attempted += 1

            soup = self.client.get_soup(url)
            if soup is None:
                stats.pages_failed += 1
                stats.failed_urls.append(url)
                failures += 1
                if failures >= config.MAX_CONSECUTIVE_PAGE_FAILURES:
                    log.error("%d consecutive page failures - stopping crawl", failures)
                    break
                url = self.guess_next_url(url)
                log.warning("Page failed; continuing with guessed next page: %s", url)
                continue

            failures = 0
            stats.pages_fetched += 1
            on_page(soup, url)
            url = self.find_next_url(soup, url)

    def scrape(self):
        result = ScrapeResult(source=self.name)

        def on_page(soup, url):
            records, errors = self.parse_page(soup, url)
            result.records.extend(records)
            result.parse_errors += errors
            log.info("Parsed %d records from %s", len(records), url)

        self.crawl(self.start_url, on_page, result)
        log.info(
            "%s: %d records, %d pages ok, %d pages failed",
            self.name, len(result.records), result.pages_fetched, result.pages_failed,
        )
        return result
```

FILE 6 of 22: scrapers/books_scraper.py

```
"""Books to Scrape: all selectors and parsing rules for this site live here."""
import logging
import re
from urllib.parse import urljoin

import config
from scrapers.base_scraper import (
    BaseScraper, ScrapeResult, attr_of, blank_record, text_of, utc_now_iso,
)

log = logging.getLogger(__name__)


class BooksScraper(BaseScraper):
    name = config.BOOKS_SOURCE
    start_url = config.BOOKS_URL

    def __init__(
        self, client, start_url=None, max_pages=None,
        fetch_categories=True, fetch_details=False,
    ):
        super().__init__(client, start_url, max_pages)
        self.fetch_categories = fetch_categories
        self.fetch_details = fetch_details
    # ---- listing pages --------------------------------------------------
```

```
    def parse_page(self, soup, page_url):
        scraped_at = utc_now_iso()
        items = soup.select("article.product_pod")
        if not items:
            log.warning("No book entries found on %s", page_url)
        records, errors = [], 0
        for item in items:
            try:
                records.append(self._parse_item(item, page_url, scraped_at))
            except Exception:  # one broken entry must not stop the page
                errors += 1
                log.warning("Skipping unparseable book on %s", page_url, exc_info=True)
        return records, errors

    def _parse_item(self, item, page_url, scraped_at):
        link = item.select_one("h3 a")
        href = attr_of(link, "href")

        rating_word = None  # rating is a CSS class: "star-rating Three"
        rating_tag = item.select_one("p.star-rating")
        if rating_tag is not None:
            words = [c for c in rating_tag.get("class", []) if c != "star-rating"]
            rating_word = words[0] if words else None

        record = blank_record(self.name)
        record.update(
            # the title attribute holds the full title; link text is truncated
            name_or_title=attr_of(link, "title") or text_of(link),
            price=text_of(item.select_one("p.price_color")),
            rating=rating_word,
            availability=text_of(item.select_one("p.availability")),
            source_url=urljoin(page_url, href) if href else None,
            scraped_at=scraped_at,
        )
        return record

    def guess_next_url(self, page_url):
        match = re.search(r"page-(\d+)\.html$", page_url)
        if match:
            return page_url[: match.start()] + f"page-{int(match.group(1)) + 1}.html"
        if page_url.endswith("index.html"):  # first page of a category
            return page_url[: -len("index.html")] + "page-2.html"
        return urljoin(page_url, "catalogue/page-2.html")  # home page

    # ---- optional enrichment ------------------------------------------------
    def scrape(self):
        result = super().scrape()
        if self.fetch_categories and result.records:
            self._attach_categories(result)
        if self.fetch_details and result.records:
            self._attach_details(result)
        return result

    def _attach_categories(self, result):
        """Category is not on listing pages, so crawl each category listing."""
        home = self.client.get_soup(self.start_url)
        if home is None:
            log.error("Could not load home page; categories left empty")
            return
        categories = []
        for link in home.select("div.side_categories ul li ul li a"):
            name, href = text_of(link), attr_of(link, "href")
            if name and href:
                categories.append((name.strip(), urljoin(self.start_url, href)))
        log.info("Found %d categories", len(categories))

        mapping = {}
        aux = ScrapeResult(source=self.name)
        for name, url in categories:
            def on_page(soup, page_url, name=name):
                for anchor in soup.select("article.product_pod h3 a"):
                    href = attr_of(anchor, "href")
                    if href:
                        mapping[urljoin(page_url, href)] = name
            self.crawl(url, on_page, aux)
```

```
        result.extra_pages_fetched += aux.pages_fetched
        result.extra_pages_failed += aux.pages_failed
        result.failed_urls.extend(aux.failed_urls)
        for record in result.records:
            record["category"] = mapping.get(record["source_url"])
        missing = sum(1 for r in result.records if r["category"] is None)
        log.info("Categories attached; %d books without a category", missing)

    def _attach_details(self, result):
        """Slow (one request per book): adds description, fills missing category."""
        for record in result.records:
            url = record["source_url"]
            if not url:
                continue
            soup = self.client.get_soup(url)
            if soup is None:
                result.extra_pages_failed += 1
                result.failed_urls.append(url)
                continue
            result.extra_pages_fetched += 1
            record["description"] = text_of(soup.select_one("#product_description + p"))
            if record["category"] is None:
                crumbs = soup.select("ul.breadcrumb li a")  # Home > Books > Category
                if len(crumbs) >= 3:
                    record["category"] = text_of(crumbs[2])
```

FILE 7 of 22: scrapers/quotes_scraper.py

```
"""Quotes to Scrape: all selectors and parsing rules for this site live here."""
import logging
import re
from urllib.parse import urljoin

import config
from scrapers.base_scraper import BaseScraper, blank_record, text_of, utc_now_iso

log = logging.getLogger(__name__)


class QuotesScraper(BaseScraper):
    name = config.QUOTES_SOURCE
    start_url = config.QUOTES_URL

    def parse_page(self, soup, page_url):
        scraped_at = utc_now_iso()
        items = soup.select("div.quote")
        if not items:
            log.warning("No quotes found on %s", page_url)
        records, errors = [], 0
        for item in items:
            try:
                record = blank_record(self.name)
                record.update(
                    name_or_title=text_of(item.select_one("span.text")),
                    author=text_of(item.select_one("small.author")),
                    tags=[tag.get_text() for tag in item.select("div.tags a.tag")],
                    source_url=page_url,  # quotes have no page of their own
                    scraped_at=scraped_at,
                )
                records.append(record)
            except Exception:
                errors += 1
                log.warning("Skipping unparseable quote on %s", page_url, exc_info=True)
        return records, errors

    def guess_next_url(self, page_url):
        match = re.search(r"/page/(\d+)/?$", page_url)
        if match:
            return page_url[: match.start()] + f"/page/{int(match.group(1)) + 1}/"
        return urljoin(page_url, "/page/2/")
```

FILE 8 of 22: processing/__init__.py

```
"""Cleaning, validation and de-duplication (no scraping logic here)."""
```

FILE 9 of 22: processing/cleaning.py

```
"""Reusable cleaning helpers. Lenient by design: strict checks live in validation."""
import re
import unicodedata
from urllib.parse import urljoin, urlsplit, urlunsplit

import config

MISSING_TOKENS = {"", "n/a", "na", "none", "null", "nan", "-", "--"}
RATING_WORDS = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5}

_WHITESPACE = re.compile(r"\s+")  # also matches non-breaking spaces
_ZERO_WIDTH = dict.fromkeys(map(ord, "\u200b\u200c\u200d\ufeff"), None)
_PRICE = re.compile(r"\d[\d,]*(?:\.\d+)?|\.\d+")
_QUOTE_PAIRS = {"\u201c": "\u201d", '"': '"'}


def clean_text(value):
    """Trim, collapse every kind of whitespace, drop zero-width chars."""
    if value is None:
        return None
    text = unicodedata.normalize("NFC", str(value)).translate(_ZERO_WIDTH)
    text = _WHITESPACE.sub(" ", text).strip()
    return text or None


def normalize_missing(value):
    """Clean text and turn placeholders such as 'N/A' or '-' into None."""
    text = clean_text(value)
    if text is None or text.casefold() in MISSING_TOKENS:
        return None
    return text


def strip_wrapping_quotes(text):
    """Remove ONE pair of surrounding quotation marks (the site wraps quotes)."""
    if text and len(text) >= 2 and _QUOTE_PAIRS.get(text[0]) == text[-1]:
        return clean_text(text[1:-1])
    return text


def parse_price(value):
    """'\u00a351.77' -> 51.77. None if missing; the cleaned text if unparseable."""
    text = normalize_missing(value)
    if text is None:
        return None
    match = _PRICE.search(text)
    if not match:
        return text  # validation will reject it as invalid_price
    number = round(float(match.group().replace(",", "")), 2)
    return -number if "-" in text[: match.start()] else number


def parse_rating(value):
    """'Three' -> 3, '4' -> 4. None if missing; cleaned text if unrecognised."""
    text = normalize_missing(value)
    if text is None:
        return None
    key = text.casefold()
    if key in RATING_WORDS:
        return RATING_WORDS[key]
    try:
        number = float(key)
    except ValueError:
        return text  # validation will reject it as invalid_rating
    return int(number) if number.is_integer() else number


def normalize_url(url, base=None):
```

```
    """Return a clean absolute http(s) URL, or None when it is not valid."""
    text = clean_text(url)
    if text is None:
        return None
    try:
        parts = urlsplit(urljoin(base, text) if base else text)
    except ValueError:
        return None
    if parts.scheme.lower() not in ("http", "https") or not parts.netloc:
        return None
    return urlunsplit(
        (parts.scheme.lower(), parts.netloc.lower(), parts.path or "/", parts.query, "")
    )


def clean_tags(values):
    """List (or comma string) -> lower-cased, de-duplicated list, order kept."""
    if values is None:
        return []
    if isinstance(values, str):
        values = values.split(",")
    seen, result = set(), []
    for value in values:
        tag = clean_text(value)
        if tag is None:
            continue
        tag = tag.casefold()
        if tag not in seen:
            seen.add(tag)
            result.append(tag)
    return result


def clean_record(raw):
    """Turn one raw scraped record into a cleaned record with the same columns."""
    source = clean_text(raw.get("source"))
    title = clean_text(raw.get("name_or_title"))
    if source == config.QUOTES_SOURCE:
        title = strip_wrapping_quotes(title)
    return {
        "source": source,
        "name_or_title": title,
        "category": normalize_missing(raw.get("category")),
        "price": parse_price(raw.get("price")),
        "rating": parse_rating(raw.get("rating")),
        "author": normalize_missing(raw.get("author")),
        "tags": clean_tags(raw.get("tags")),
        "availability": normalize_missing(raw.get("availability")),
        "description": normalize_missing(raw.get("description")),
        "source_url": normalize_url(raw.get("source_url")),
        "scraped_at": clean_text(raw.get("scraped_at")),
    }
```

FILE 10 of 22: processing/validation.py

```
"""Strict record checks. Rejected records are kept with their reasons."""
import math
from collections import Counter

import config
from processing.cleaning import normalize_url


def _is_number(value):
    return (
        isinstance(value, (int, float))
        and not isinstance(value, bool)
        and math.isfinite(value)
    )


def validate_record(record):
    """Return a list of reason codes. An empty list means the record is valid."""
    source = record.get("source")
    if source not in config.KNOWN_SOURCES:
```

```
        return ["unknown_source"]

    reasons = []
    for name in config.REQUIRED_FIELDS[source]:
        if record.get(name) in (None, ""):
            reasons.append(f"missing_{name}")

    price = record.get("price")
    if price is not None and not (_is_number(price) and price >= 0):
        reasons.append("invalid_price")

    rating = record.get("rating")
    if rating is not None:
        in_range = _is_number(rating) and config.RATING_MIN <= rating <= config.RATING_MAX
        if not in_range:
            reasons.append("invalid_rating")

    url = record.get("source_url")
    if url and normalize_url(url) is None:
        reasons.append("invalid_source_url")
    return reasons


def validate_records(records):
    """Split records into (valid, rejected). rejected = [{record, reasons}]."""
    valid, rejected = [], []
    for record in records:
        reasons = validate_record(record)
        if reasons:
            rejected.append({"record": record, "reasons": reasons})
        else:
            valid.append(record)
    return valid, rejected


def count_reasons(rejected):
    return Counter(reason for item in rejected for reason in item["reasons"])
```

FILE 11 of 22: processing/deduplication.py

```
"""Duplicate detection based on normalised key fields (see config.DEDUP_KEYS)."""
import hashlib
import re
import unicodedata

import config

_NON_WORD = re.compile(r"[\W_]+")


def normalize_for_key(value):
    """Case-fold, NFKC-normalise, drop punctuation and collapse spaces.

    '  Example Book Title ', 'EXAMPLE BOOK TITLE' and 'Example, Book Title!'
    all become 'example book title'.
    """
    if value is None:
        return ""
    text = unicodedata.normalize("NFKC", str(value)).casefold()
    return _NON_WORD.sub(" ", text).strip()


def record_key(record):
    """SHA-256 fingerprint of the source plus the source's key fields."""
    fields = config.DEDUP_KEYS.get(record.get("source"), ("name_or_title",))
    parts = [str(record.get("source"))]
    parts += [normalize_for_key(record.get(name)) for name in fields]
    return hashlib.sha256("\x1f".join(parts).encode("utf-8")).hexdigest()


def deduplicate(records):
    """Keep the first record of each key. Returns (unique, duplicates).

    Each duplicate entry says which kept record it duplicates, so removals
    can be audited.
```

```
    """
    first_seen, unique, duplicates = {}, [], []
    for record in records:
        key = record_key(record)
        if key in first_seen:
            duplicates.append(
                {
                    "record": record,
                    "duplicate_of": first_seen[key]["source_url"],
                    "key": key,
                }
            )
        else:
            first_seen[key] = record
            unique.append(record)
    return unique, duplicates
```

FILE 12 of 22: pipeline.py

```
"""Orchestrates: scrape -> clean -> validate -> deduplicate -> consolidate -> save."""
import csv
import json
import logging
import time
from collections import Counter
from datetime import datetime, timezone
from pathlib import Path

import config
from processing.cleaning import clean_record
from processing.deduplication import deduplicate
from processing.validation import validate_records
from scrapers.base_scraper import ScrapeResult

log = logging.getLogger(__name__)


def consolidate(records):
    """One common row layout for every source (CSV-ready, None -> empty)."""
    rows = []
    for record in records:
        row = {name: record.get(name) for name in config.SCHEMA}
        row["tags"] = config.TAG_SEPARATOR.join(record.get("tags") or [])
        rows.append({k: ("" if v is None else v) for k, v in row.items()})
    return rows


def write_csv(rows, path):
    # utf-8-sig so Excel shows pound signs and accents correctly
    with open(path, "w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.DictWriter(handle, fieldnames=config.SCHEMA)
        writer.writeheader()
        writer.writerows(rows)


def count_csv_rows(path):
    with open(path, newline="", encoding="utf-8-sig") as handle:
        return sum(1 for _ in csv.DictReader(handle))


def _source_stats(result, cleaned, rejected, duplicates, unique, clean_failures):
    name = result.source
    rejected_here = [r for r in rejected if r["record"].get("source") == name]
    reasons = Counter(x for r in rejected_here for x in r["reasons"])
    return {
        "pages_fetched": result.pages_fetched,
        "pages_failed": result.pages_failed,
        "supporting_pages_fetched": result.extra_pages_fetched,
        "supporting_pages_failed": result.extra_pages_failed,
        "failed_urls": result.failed_urls,
        "parse_errors": result.parse_errors,
        "scraper_error": result.error,
        "scraped": len(result.records),
        "cleaned": len(result.records) - clean_failures[name],
        "rejected": len(rejected_here),
```

```
        "rejection_reasons": dict(reasons),
        "duplicates_removed": sum(
            1 for d in duplicates if d["record"].get("source") == name
        ),
        "final": sum(1 for r in unique if r.get("source") == name),
    }


def run_pipeline(scrapers, output_dir, run_args=None):
    started = datetime.now(timezone.utc)
    timer = time.perf_counter()
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    # 1. Scrape - a crashing source must not stop the other source
    results = []
    for scraper in scrapers:
        log.info("=== Scraping %s ===", scraper.name)
        try:
            results.append(scraper.scrape())
        except Exception as exc:
            log.exception("Scraper %s crashed; continuing", scraper.name)
            results.append(ScrapeResult(source=scraper.name, error=repr(exc)))

    # 2. Clean
    log.info("=== Cleaning ===")
    cleaned, clean_rejected = [], []
    clean_failures = Counter()
    for result in results:
        for raw in result.records:
            try:
                cleaned.append(clean_record(raw))
            except Exception:
                log.exception("Cleaning failed for %s", raw.get("source_url"))
                clean_failures[result.source] += 1
                clean_rejected.append({"record": raw, "reasons": ["cleaning_error"]})

    # 3. Validate
    log.info("=== Validating %d records ===", len(cleaned))
    valid, rejected = validate_records(cleaned)
    rejected.extend(clean_rejected)
    for item in rejected:
        log.warning("Rejected %s: %s", item["record"].get("source_url"), item["reasons"])

    # 4. Deduplicate
    log.info("=== Deduplicating %d valid records ===", len(valid))
    unique, duplicates = deduplicate(valid)
    for item in duplicates:
        log.info("Duplicate removed: %r", item["record"].get("name_or_title"))

    # 5. Consolidate + save
    rows = consolidate(unique)
    csv_path = output_dir / "final_dataset.csv"
    write_csv(rows, csv_path)
    csv_rows = count_csv_rows(csv_path)

    sources = {
        r.source: _source_stats(r, cleaned, rejected, duplicates, unique, clean_failures)
        for r in results
    }
    totals = {
        key: sum(s[key] for s in sources.values())
        for key in ("scraped", "cleaned", "rejected", "duplicates_removed", "final")
    }
    reconciles = all(
        s["scraped"] == s["rejected"] + s["duplicates_removed"] + s["final"]
        for s in sources.values()
    )
    finished = datetime.now(timezone.utc)
    summary = {
        "run": {
            "started_at": started.isoformat(timespec="seconds"),
            "finished_at": finished.isoformat(timespec="seconds"),
            "duration_seconds": round(time.perf_counter() - timer, 2),
            "arguments": run_args or {},
```

```
        },
        "sources": sources,
        "totals": totals,
        "checks": {
            "scraped_equals_rejected_plus_duplicates_plus_final": reconciles,
            "csv_rows": csv_rows,
            "csv_rows_match_final_total": csv_rows == totals["final"] == len(unique),
        },
    }
    with open(output_dir / "summary_report.json", "w", encoding="utf-8") as handle:
        json.dump(summary, handle, indent=2, ensure_ascii=False)
    with open(output_dir / "excluded_records.json", "w", encoding="utf-8") as handle:
        json.dump(
            {"rejected": rejected, "duplicates": duplicates},
            handle, indent=2, ensure_ascii=False,
        )
    log.info("Done: %s", totals)
    return summary
```

FILE 13 of 22: main.py

```
"""Entry point:  python main.py   (see README.md for options)."""
import argparse
import logging
import sys
from datetime import datetime
from pathlib import Path

import config
from pipeline import run_pipeline
from scrapers.books_scraper import BooksScraper
from scrapers.http_client import HttpClient
from scrapers.quotes_scraper import QuotesScraper


def parse_args(argv=None):
    parser = argparse.ArgumentParser(
        description="Scrape Books to Scrape and Quotes to Scrape into one dataset."
    )
    parser.add_argument("--sources", nargs="+", choices=["books", "quotes"],
                        default=["books", "quotes"], help="which sources to scrape")
    parser.add_argument("--max-pages", type=int, default=None,
                        help="limit pages per crawl (quick trial runs)")
    parser.add_argument("--no-categories", action="store_true",
                        help="skip the extra category crawl for books")
    parser.add_argument("--with-details", action="store_true",
                        help="fetch every book page for its description (slow)")
    parser.add_argument("--delay", type=float, default=config.DEFAULT_DELAY,
                        help="seconds between requests (default 0.5)")
    parser.add_argument("--output-dir", default=str(config.DEFAULT_OUTPUT_DIR))
    parser.add_argument("--log-dir", default=str(config.DEFAULT_LOG_DIR))
    parser.add_argument("--log-level", default="INFO",
                        choices=["DEBUG", "INFO", "WARNING", "ERROR"])
    args = parser.parse_args(argv)
    if args.max_pages is not None and args.max_pages < 1:
        parser.error("--max-pages must be at least 1")
    if args.delay < 0:
        parser.error("--delay cannot be negative")
    return args


def setup_logging(log_dir, level):
    Path(log_dir).mkdir(parents=True, exist_ok=True)
    log_path = Path(log_dir) / f"scraper_{datetime.now():%Y%m%d_%H%M%S}.log"
    logging.basicConfig(
        level=getattr(logging, level),
        format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
        handlers=[logging.FileHandler(log_path, encoding="utf-8"),
                  logging.StreamHandler()],
        force=True,
    )
    return log_path


def main(argv=None):
```

```
    args = parse_args(argv)
    log_path = setup_logging(args.log_dir, args.log_level)
    client = HttpClient(delay=args.delay)

    scrapers = []
    if "books" in args.sources:
        scrapers.append(BooksScraper(
            client, max_pages=args.max_pages,
            fetch_categories=not args.no_categories,
            fetch_details=args.with_details,
        ))
    if "quotes" in args.sources:
        scrapers.append(QuotesScraper(client, max_pages=args.max_pages))

    summary = run_pipeline(scrapers, args.output_dir, run_args=vars(args))

    print("\n=== Summary ===")
    for name, stats in summary["sources"].items():
        print(f"{name}: scraped={stats['scraped']} rejected={stats['rejected']} "
              f"duplicates={stats['duplicates_removed']} final={stats['final']} "
              f"(pages ok/failed: {stats['pages_fetched']}/{stats['pages_failed']})")
    print(f"Total final records: {summary['totals']['final']}")
    print(f"Checks: {summary['checks']}")
    print(f"Output: {args.output_dir}\nLog:    {log_path}")
    return 0 if summary["totals"]["final"] > 0 else 1


if __name__ == "__main__":
    sys.exit(main())
```

FILE 14 of 22: conftest.py

```
# Intentionally empty. Its presence lets pytest (optional) import project modules.
```

FILE 15 of 22: tests/helpers.py

```
"""Fake HTML pages and a fake HTTP client so tests never touch the internet."""
from bs4 import BeautifulSoup


class FakeClient:
    """Mimics HttpClient.get_soup using an in-memory {url: html} mapping."""

    def __init__(self, pages):
        self.pages = pages
        self.requested = []

    def get_soup(self, url):
        self.requested.append(url)
        html = self.pages.get(url)
        return BeautifulSoup(html, "lxml") if html else None


def book_item(title, href, price="\u00a351.77", rating="Three", avail="In stock"):
    price_html = f'<p class="price_color">{price}</p>' if price else ""
    return (
        '<article class="product_pod">'
        f'<p class="star-rating {rating}"></p>'
        f'<h3><a href="{href}" title="{title}">{title[:12]}...</a></h3>'
        f'<div class="product_price">{price_html}'
        f'<p class="instock availability">\n  {avail}\n</p></div></article>'
    )


def books_page(items, next_href=None, sidebar=""):
    nxt = f'<li class="next"><a href="{next_href}">next</a></li>' if next_href else ""
    return (
        f"<html><body>{sidebar}<ol>{''.join(items)}</ol>"
        f'<ul class="pager">{nxt}</ul></body></html>'
    )


def quote_item(text, author, tags):
    tag_html = "".join(f'<a class="tag" href="/tag/{t}/">{t}</a>' for t in tags)
```

```
    return (
        '<div class="quote">'
        f'<span class="text">\u201c{text}\u201d</span>'
        f'<span>by <small class="author">{author}</small></span>'
        f'<div class="tags">{tag_html}</div></div>'
    )


def quotes_page(items, next_href=None):
    nxt = f'<li class="next"><a href="{next_href}">Next</a></li>' if next_href else ""
    return f'<html><body>{"".join(items)}<ul class="pager">{nxt}</ul></body></html>'
```

FILE 16 of 22: tests/test_cleaning.py

```
import unittest

from processing import cleaning as c


class CleaningTests(unittest.TestCase):
    def test_clean_text_collapses_whitespace_and_nbsp(self):
        self.assertEqual(c.clean_text("  A\u00a0 Light \n in\tthe Attic  "),
                         "A Light in the Attic")
        self.assertIsNone(c.clean_text("   "))
        self.assertIsNone(c.clean_text(None))

    def test_normalize_missing(self):
        for value in ("N/A", " none ", "-", "", None):
            self.assertIsNone(c.normalize_missing(value))
        self.assertEqual(c.normalize_missing(" Poetry "), "Poetry")

    def test_parse_price(self):
        self.assertEqual(c.parse_price("\u00a351.77"), 51.77)
        self.assertEqual(c.parse_price("\u00c2\u00a351.77"), 51.77)  # mojibake
        self.assertEqual(c.parse_price(" \u00a3 1,234.50 "), 1234.5)
        self.assertEqual(c.parse_price("-\u00a35.00"), -5.0)
        self.assertIsNone(c.parse_price(None))
        self.assertEqual(c.parse_price("free"), "free")  # left for validation

    def test_parse_rating(self):
        self.assertEqual(c.parse_rating("Three"), 3)
        self.assertEqual(c.parse_rating(" FIVE "), 5)
        self.assertEqual(c.parse_rating("4.0"), 4)
        self.assertIsNone(c.parse_rating(None))
        self.assertEqual(c.parse_rating("Seven"), "Seven")

    def test_normalize_url(self):
        self.assertEqual(
            c.normalize_url("../a/index.html", "https://Books.toscrape.com/x/y.html"),
            "https://books.toscrape.com/a/index.html",
        )
        self.assertIsNone(c.normalize_url("javascript:void(0)"))
        self.assertIsNone(c.normalize_url("not a url"))
        self.assertIsNone(c.normalize_url(None))

    def test_clean_tags(self):
        self.assertEqual(c.clean_tags([" Love ", "love", "", "Life "]), ["love", "life"])
        self.assertEqual(c.clean_tags(None), [])

    def test_strip_wrapping_quotes_only_removes_one_pair(self):
        self.assertEqual(c.strip_wrapping_quotes("\u201cHello.\u201d"), "Hello.")
        self.assertEqual(c.strip_wrapping_quotes("It's fine"), "It's fine")

    def test_clean_record_quote(self):
        raw = {"source": "Quotes to Scrape", "name_or_title": "\u201c Hi  there \u201d",
               "author": " Jane ", "tags": ["A", "a"], "price": None,
               "source_url": "https://quotes.toscrape.com/page/2/",
               "scraped_at": "2026-01-01T00:00:00+00:00"}
        cleaned = c.clean_record(raw)
        self.assertEqual(cleaned["name_or_title"], "Hi there")
        self.assertEqual(cleaned["author"], "Jane")
        self.assertEqual(cleaned["tags"], ["a"])
        self.assertIsNone(cleaned["price"])
if __name__ == "__main__":
    unittest.main()
```

FILE 17 of 22: tests/test_validation_dedup.py

```
import unittest

import config
from processing.deduplication import deduplicate, normalize_for_key, record_key
from processing.validation import validate_record, validate_records


def book(**overrides):
    record = {name: None for name in config.SCHEMA}
    record.update(
        source=config.BOOKS_SOURCE, name_or_title="Example Book Title", price=10.0,
        rating=3, source_url="https://books.toscrape.com/catalogue/x_1/index.html",
        scraped_at="2026-01-01T00:00:00+00:00", tags=[],
    )
    record.update(overrides)
    return record


def quote(**overrides):
    record = {name: None for name in config.SCHEMA}
    record.update(
        source=config.QUOTES_SOURCE, name_or_title="Be yourself.", author="Oscar Wilde",
        source_url="https://quotes.toscrape.com/", tags=["life"],
        scraped_at="2026-01-01T00:00:00+00:00",
    )
    record.update(overrides)
    return record


class ValidationTests(unittest.TestCase):
    def test_valid_records(self):
        self.assertEqual(validate_record(book()), [])
        self.assertEqual(validate_record(quote()), [])

    def test_rejections(self):
        self.assertEqual(validate_record(book(source="Other")), ["unknown_source"])
        self.assertIn("missing_name_or_title", validate_record(book(name_or_title=None)))
        self.assertIn("missing_price", validate_record(book(price=None)))
        self.assertIn("invalid_price", validate_record(book(price="free")))
        self.assertIn("invalid_price", validate_record(book(price=-1.0)))
        self.assertIn("invalid_rating", validate_record(book(rating=9)))
        self.assertIn("invalid_rating", validate_record(book(rating="Seven")))
        self.assertIn("missing_source_url", validate_record(book(source_url=None)))
        self.assertIn("invalid_source_url", validate_record(book(source_url="ftp://x")))
        self.assertIn("missing_author", validate_record(quote(author=None)))

    def test_validate_records_splits(self):
        valid, rejected = validate_records([book(), book(price=None)])
        self.assertEqual(len(valid), 1)
        self.assertEqual(rejected[0]["reasons"], ["missing_price"])


class DeduplicationTests(unittest.TestCase):
    def test_normalize_for_key(self):
        variants = ["Example Book Title", " Example Book Title ",
                    "EXAMPLE BOOK TITLE", "Example, Book  Title!"]
        self.assertEqual({normalize_for_key(v) for v in variants},
                         {"example book title"})

    def test_equivalent_books_are_duplicates(self):
        records = [book(), book(name_or_title="  EXAMPLE book title "),
                   book(name_or_title="Another Book")]
        unique, duplicates = deduplicate(records)
        self.assertEqual(len(unique), 2)
        self.assertEqual(len(duplicates), 1)

    def test_same_title_different_price_is_kept(self):
        unique, duplicates = deduplicate([book(), book(price=11.0)])
        self.assertEqual((len(unique), len(duplicates)), (2, 0))

    def test_quotes_use_text_and_author(self):
        records = [quote(), quote(name_or_title="BE YOURSELF"), quote(author="Other")]
        unique, duplicates = deduplicate(records)
```

```
        self.assertEqual((len(unique), len(duplicates)), (2, 1))

    def test_sources_never_collide(self):
        self.assertNotEqual(record_key(book(name_or_title="x", price=None)),
                            record_key(quote(name_or_title="x", author=None)))


if __name__ == "__main__":
    unittest.main()
```

FILE 18 of 22: tests/test_scrapers.py

```
import unittest

from helpers import FakeClient, book_item, books_page, quote_item, quotes_page
from scrapers.books_scraper import BooksScraper
from scrapers.quotes_scraper import QuotesScraper

HOME = "https://books.test/"
P2 = "https://books.test/catalogue/page-2.html"
P3 = "https://books.test/catalogue/page-3.html"


def three_page_books_site(skip=()):
    pages = {
        HOME: books_page(
            [book_item("Book One", "catalogue/one_1/index.html")],
            "catalogue/page-2.html"),
        P2: books_page(
            [book_item("Book Two", "two_2/index.html", rating="Five")],
            "page-3.html"),
        P3: books_page([book_item("Book Three", "three_3/index.html")]),
    }
    return {url: html for url, html in pages.items() if url not in skip}


class BooksScraperTests(unittest.TestCase):
    def test_follows_next_links_to_the_last_page(self):
        client = FakeClient(three_page_books_site())
        result = BooksScraper(client, start_url=HOME, fetch_categories=False).scrape()
        self.assertEqual(result.pages_fetched, 3)
        self.assertEqual(
            [r["name_or_title"] for r in result.records],
            ["Book One", "Book Two", "Book Three"])
        self.assertEqual(result.records[1]["source_url"],
                         "https://books.test/catalogue/two_2/index.html")
        self.assertEqual(result.records[1]["rating"], "Five")  # raw value

    def test_failed_page_is_skipped_and_crawl_continues(self):
        client = FakeClient(three_page_books_site(skip=(P2,)))
        result = BooksScraper(client, start_url=HOME, fetch_categories=False).scrape()
        self.assertEqual(result.pages_failed, 1)
        self.assertEqual(result.failed_urls, [P2])
        self.assertEqual([r["name_or_title"] for r in result.records],
                         ["Book One", "Book Three"])

    def test_two_consecutive_failures_stop_the_crawl(self):
        client = FakeClient(three_page_books_site(skip=(P2, P3)))
        result = BooksScraper(client, start_url=HOME, fetch_categories=False).scrape()
        self.assertEqual(result.pages_failed, 2)
        self.assertEqual(len(result.records), 1)

    def test_max_pages(self):
        client = FakeClient(three_page_books_site())
        result = BooksScraper(
            client, start_url=HOME, max_pages=2, fetch_categories=False).scrape()
        self.assertEqual(result.pages_fetched, 2)

    def test_missing_elements_do_not_crash(self):
        page = books_page([book_item("No Price", "np_1/index.html", price=None)])
        client = FakeClient({HOME: page})
        result = BooksScraper(client, start_url=HOME, fetch_categories=False).scrape()
        self.assertEqual(len(result.records), 1)
        self.assertIsNone(result.records[0]["price"])
    def test_category_enrichment(self):
```

```
        sidebar = (
            '<div class="side_categories"><ul><li><a href="x">Books</a><ul>'
            '<li><a href="catalogue/category/books/travel_2/index.html">'
            "\n  Travel\n</a></li></ul></li></ul></div>")
        home = books_page(
            [book_item("Book One", "catalogue/one_1/index.html")], sidebar=sidebar)
        category = books_page(
            [book_item("Book One", "../../../one_1/index.html")])
        client = FakeClient({
            HOME: home,
            "https://books.test/catalogue/category/books/travel_2/index.html": category,
        })
        result = BooksScraper(client, start_url=HOME).scrape()
        self.assertEqual(result.records[0]["category"], "Travel")
        self.assertEqual(result.extra_pages_fetched, 1)


class QuotesScraperTests(unittest.TestCase):
    def test_parses_quotes_and_paginates(self):
        home, page2 = "https://quotes.test/", "https://quotes.test/page/2/"
        client = FakeClient({
            home: quotes_page([quote_item("Be yourself.", "Oscar Wilde", ["life", "x"])],
                              "/page/2/"),
            page2: quotes_page([quote_item("Stay hungry.", "Steve Jobs", [])]),
        })
        result = QuotesScraper(client, start_url=home).scrape()
        self.assertEqual(result.pages_fetched, 2)
        self.assertEqual(len(result.records), 2)
        first = result.records[0]
        self.assertEqual(first["author"], "Oscar Wilde")
        self.assertEqual(first["tags"], ["life", "x"])
        self.assertEqual(result.records[1]["source_url"], page2)
        self.assertIsNone(first["price"])  # not applicable -> None, not invented

    def test_failed_first_page_uses_guessed_next_page(self):
        page2 = "https://quotes.test/page/2/"
        client = FakeClient(
            {page2: quotes_page([quote_item("Hi.", "Someone", ["a"])])})
        result = QuotesScraper(client, start_url="https://quotes.test/").scrape()
        self.assertEqual(result.pages_failed, 1)
        self.assertEqual(len(result.records), 1)


if __name__ == "__main__":
    unittest.main()
```

FILE 19 of 22: tests/test_http_client.py

```
"""Runs HttpClient against a tiny local server (no internet needed)."""
import socket
import threading
import unittest
from http.server import BaseHTTPRequestHandler, HTTPServer

from scrapers.http_client import HttpClient


class Handler(BaseHTTPRequestHandler):
    hits = {}

    def do_GET(self):
        Handler.hits[self.path] = Handler.hits.get(self.path, 0) + 1
        if self.path == "/flaky" and Handler.hits[self.path] < 3:
            self.send_response(503)
        elif self.path == "/missing":
            self.send_response(404)
        else:
            self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write("<p>\u00a351.77</p>".encode("utf-8"))

    def log_message(self, *args):
        pass
class HttpClientTests(unittest.TestCase):
```

```
    @classmethod
    def setUpClass(cls):
        cls.server = HTTPServer(("127.0.0.1", 0), Handler)
        cls.base = f"http://127.0.0.1:{cls.server.server_port}"
        threading.Thread(target=cls.server.serve_forever, daemon=True).start()

    @classmethod
    def tearDownClass(cls):
        cls.server.shutdown()
        cls.server.server_close()

    def setUp(self):
        Handler.hits = {}
        self.client = HttpClient(delay=0, timeout=2, max_retries=3, backoff=0.01)

    def test_success_decodes_utf8(self):
        self.assertIn("\u00a351.77", self.client.get_html(self.base + "/ok"))

    def test_retries_5xx_then_succeeds(self):
        self.assertIsNotNone(self.client.get_html(self.base + "/flaky"))
        self.assertEqual(Handler.hits["/flaky"], 3)

    def test_404_is_not_retried(self):
        self.assertIsNone(self.client.get_html(self.base + "/missing"))
        self.assertEqual(Handler.hits["/missing"], 1)

    def test_connection_refused_returns_none(self):
        sock = socket.socket()
        sock.bind(("127.0.0.1", 0))
        port = sock.getsockname()[1]
        sock.close()  # nothing is listening on this port now
        self.assertIsNone(self.client.get_html(f"http://127.0.0.1:{port}/"))

    def test_get_soup_returns_none_on_failure(self):
        self.assertIsNone(self.client.get_soup(self.base + "/missing"))


if __name__ == "__main__":
    unittest.main()
```

FILE 20 of 22: tests/test_pipeline.py

```
import csv
import json
import tempfile
import unittest
from pathlib import Path

from helpers import FakeClient, book_item, books_page, quote_item, quotes_page
from pipeline import run_pipeline
from scrapers.books_scraper import BooksScraper
from scrapers.quotes_scraper import QuotesScraper


class PipelineTests(unittest.TestCase):
    def test_end_to_end_counts_add_up(self):
        pages = {
            "https://books.test/": books_page([
                book_item("Example Book Title", "catalogue/a_1/index.html"),
                book_item("  EXAMPLE BOOK title ", "catalogue/a_2/index.html"),  # dup
                book_item("Cheap Book", "catalogue/b_3/index.html", price=None),  # bad
                book_item("Odd Rating", "catalogue/c_4/index.html", rating="Nine"),
                book_item("Good Book", "catalogue/d_5/index.html", price="\u00a312.50"),
            ]),
            "https://quotes.test/": quotes_page([
                quote_item("Be yourself.", "Oscar Wilde", ["Life"]),
                quote_item("be  yourself", "oscar wilde", ["life"]),  # dup
                quote_item("No author here.", "", []),                # bad
            ]),
        }
        client = FakeClient(pages)
        scrapers = [
            BooksScraper(client, start_url="https://books.test/",
                         fetch_categories=False),
            QuotesScraper(client, start_url="https://quotes.test/"),
```

```
        ]
        with tempfile.TemporaryDirectory() as tmp:
            summary = run_pipeline(scrapers, tmp)

            books = summary["sources"]["Books to Scrape"]
            quotes = summary["sources"]["Quotes to Scrape"]
            self.assertEqual((books["scraped"], books["rejected"],
                              books["duplicates_removed"], books["final"]),
                             (5, 2, 1, 2))
            self.assertEqual((quotes["scraped"], quotes["rejected"],
                              quotes["duplicates_removed"], quotes["final"]),
                             (3, 1, 1, 1))
            self.assertEqual(books["rejection_reasons"],
                             {"missing_price": 1, "invalid_rating": 1})
            self.assertTrue(summary["checks"]["csv_rows_match_final_total"])
            self.assertTrue(
                summary["checks"]["scraped_equals_rejected_plus_duplicates_plus_final"])

            with open(Path(tmp) / "final_dataset.csv", encoding="utf-8-sig",
                      newline="") as handle:
                rows = list(csv.DictReader(handle))
            self.assertEqual(len(rows), 3)
            self.assertEqual({r["source"] for r in rows},
                             {"Books to Scrape", "Quotes to Scrape"})
            good = next(r for r in rows if r["name_or_title"] == "Good Book")
            self.assertEqual((good["price"], good["rating"]), ("12.5", "3"))
            self.assertEqual(good["author"], "")  # not applicable -> empty

            for name in ("summary_report.json", "excluded_records.json"):
                self.assertTrue(json.loads((Path(tmp) / name).read_text("utf-8")))

    def test_crashing_source_does_not_stop_the_other(self):
        class Boom:
            name = "Broken Source"

            def scrape(self):
                raise RuntimeError("boom")

        client = FakeClient({"https://quotes.test/": quotes_page(
            [quote_item("Hello there.", "Someone", ["a"])])})
        scrapers = [Boom(), QuotesScraper(client, start_url="https://quotes.test/")]
        with tempfile.TemporaryDirectory() as tmp:
            summary = run_pipeline(scrapers, tmp)
        self.assertIn("boom", summary["sources"]["Broken Source"]["scraper_error"])
        self.assertEqual(summary["totals"]["final"], 1)


if __name__ == "__main__":
    unittest.main()
```

FILE 21 of 22: README.md

```
# Multi-Source Web Scraping and Data Consolidation

A small ETL pipeline that scrapes two practice sites, cleans and validates the data,
removes duplicates and writes ONE consolidated CSV plus a JSON summary report.

    Scrape (both sites) -> Clean -> Validate -> Deduplicate -> Consolidate -> Save

Sources: https://books.toscrape.com/ (1,000 books, 50 pages) and
https://quotes.toscrape.com/ (100 quotes, 10 pages).

## Python version
Python 3.10, 3.11 or 3.12.

## Setup
    python -m venv .venv
    # Windows:      .venv\Scripts\activate
    # macOS/Linux:  source .venv/bin/activate
    pip install -r requirements.txt

## Dependencies
requests (HTTP), beautifulsoup4 (HTML search), lxml (fast HTML parser).
Everything else is the standard library (csv, json, logging, re, urllib.parse,
hashlib, unicodedata, argparse, unittest ...). No accounts or API keys.
```

```
## How to run
    python main.py                      # full run, ~1,100 records, about 2-4 minutes
    python main.py --max-pages 2        # quick trial run
    python main.py --sources quotes     # one source only
    python main.py --no-categories      # skip the extra category crawl for books
    python main.py --with-details       # also fetch each book page for its description
                                        # (1,000 extra requests, ~10 minutes)
    python main.py --delay 1.0 --log-level DEBUG

Run the tests (offline, no internet needed):

    python -m unittest discover -s tests -v

Outputs (created in `output/` and `logs/`):

| File                          | Content                                              |
|-------------------------------|------------------------------------------------------|
| output/final_dataset.csv      | one row per unique, valid record, same columns       |
| output/summary_report.json    | per-source counts, reasons, duration, self-checks    |
| output/excluded_records.json  | every rejected / duplicate record with the reason    |
| logs/scraper_<timestamp>.log  | every request, warning, retry and error              |

## Step 1 - Observations about the sources
* Books: 20 books per listing page, "Page 1 of 50", 1,000 results. A relative
  `li.next a` link leads to the next page (`catalogue/page-2.html`, then `page-3.html`
  ... resolved against the CURRENT page URL with `urljoin`).
* Books: title text is truncated ("A Light in the ...") but the link's `title`
  attribute has the full title. Price is `p.price_color` (pound sign + 51.77). The rating is a
  CSS class (`star-rating Three`). Availability is `p.availability`.
* Books: the category is NOT on listing pages. It is on each book page or on the
  50 category listing pages (sidebar). We crawl the category pages (~65 requests)
  instead of 1,000 book pages.
* Quotes: 10 quotes per page in `div.quote`: `span.text` (wrapped in curly quotes),
  `small.author`, `a.tag`. Same `li.next a` pagination (`/page/2/`).
* The sites share nothing: a book has price/rating/category, a quote has
  author/tags. That is why a common schema with nullable columns is needed.

## Data model (common schema)
`source, name_or_title, category, price, rating, author, tags, availability,
description, source_url, scraped_at`

* `name_or_title` holds the book title or the quote text, so both fit one column.
* Fields that do not apply to a source stay empty (None / empty CSV cell). Nothing is
  invented: a quote has no price, a book has no author.
* `tags` is joined with `|`. `price` is in GBP (the site's currency).
* `source_url` is the book's own page; quotes have no page of their own, so it is the
  listing page where the quote appeared. `scraped_at` is UTC ISO-8601.
* `description` is only filled with `--with-details`.

## How pagination works
`BaseScraper.crawl()` starts at the first page, parses it, finds the `li.next a`
link, resolves it to a full URL and repeats until there is no next link. No page
number is hard-coded. A `visited` set prevents infinite loops.

If a page fails after all retries, its "next" link is unknown, so the scraper guesses
the next URL from the site's pattern (`page-N.html`, `/page/N/`) and continues. After
2 consecutive failures the crawl for that source stops.

## Cleaning approach (processing/cleaning.py)
* Whitespace: trim and collapse spaces, tabs, newlines and non-breaking spaces;
  drop zero-width characters.
* Placeholders such as "N/A", "none", "-" become None.
* Price: pound-sign text such as "51.77" -> 51.77 (float). Rating words:
  "Three" -> 3.
* Quote text: one pair of surrounding curly quotes is removed.
* Tags: trimmed, lower-cased, de-duplicated, order kept.
* URLs: made absolute, scheme and host lower-cased, fragment removed; non-http(s) or
  host-less URLs become None.
* Cleaning is lenient: a value that cannot be parsed (price "free") is passed on
  unchanged so that validation can reject it with a clear reason.

## Validation approach (processing/validation.py)
Each record gets a list of reason codes; an empty list means valid.
* `unknown_source`
* `missing_<field>` for required fields: books need title, price, rating, URL,
```

```
  timestamp; quotes need text, author, URL, timestamp.
* `invalid_price` (not a number or negative), `invalid_rating` (not a number in 1-5),
  `invalid_source_url`.
Rejected records are never dropped silently: they are counted per reason in the summary
and saved in `excluded_records.json`.

## Deduplication approach (processing/deduplication.py)
Exact comparison is not enough ("Example Book Title", " Example Book Title ",
"EXAMPLE BOOK TITLE"). Each record gets a fingerprint:

    SHA-256( source + normalised key fields )

Key fields (`config.DEDUP_KEYS`): books = title + price, quotes = text + author.
Normalisation = Unicode NFKC, case-fold, punctuation removed, spaces collapsed.
So "Example, Book  Title!" equals "example book title". The source is part of the
fingerprint, so a book and a quote can never collide. Price is part of the book key so
two different books that happen to share a title are not merged.

Strategy: the FIRST record is kept and later ones are removed (not just flagged),
because the final dataset must be one row per unique item. Removals stay auditable:
`excluded_records.json` lists each removed record and which kept record it duplicates.
The live practice sites contain no duplicates, so a real run is expected to report 0;
the logic is proven by the unit tests, which use deliberately messy duplicates.

## Error-handling approach
* HTTP (scrapers/http_client.py): timeout on every request; retries (3 attempts,
  exponential backoff 1s, 2s, 4s) for timeouts, connection errors and HTTP
  429/500/502/503/504; other errors such as 404 are not retried. The client returns
  None instead of raising.
* Pages: a failed page is logged and skipped; the crawl continues (see pagination).
* Elements: missing HTML elements give None values (`text_of`, `attr_of`), and each
  item is parsed inside its own try/except so one bad entry cannot stop a page.
* Sources: if a scraper crashes, the other source still runs.
* Data: bad values are rejected with reasons, never crash the run.
* Rate limiting: at least 0.5 s between requests (`--delay`).
* Logging: console + `logs/scraper_<timestamp>.log`.

## Self-checks in the summary report
* `scraped == rejected + duplicates_removed + final` for every source.
* CSV row count (re-read from disk) equals the final total in the JSON.

## Assumptions
* The two sites keep their current HTML structure.
* Book price is in GBP and book availability is the text shown on the listing page.
* Two quotes with the same text and author are the same quote.

## Known limitations
* Sequential scraping (polite, 2-4 minutes); no async or parallel requests.
* No checkpoint/resume: an interrupted run starts again.
* Quote `source_url` is the listing page, not a unique URL per quote.
* Author biography pages of the quotes site are not scraped.
* Pages are parsed with CSS selectors; a redesign of the sites would need selector
  updates in `scrapers/books_scraper.py` and `scrapers/quotes_scraper.py` only.

## If this had to run in production
Scheduler (cron/Airflow), checkpointing and incremental runs, a database instead of
CSV, metrics/alerts on rejected and failed counts, and a proxy-aware async client.

## AI usage summary
See AI_USAGE.md.
```

FILE 22 of 22: AI_USAGE.md

```
# AI Usage

## Tools used
* Claude (Anthropic) - generated the first full draft of the project from the
  assignment PDF.

## What it was used for
* Project structure, data model and pipeline design.
* Scrapers, cleaning, validation, deduplication, logging and retry code.
* Unit tests and the README.

## Representative prompt
```

"Create a project based on the company requirements (assignment PDF). Give the code
in one PDF file that I can run in VS Code."

AI-assisted parts
All of the code, tests and documentation in this submission was generated with AI
assistance. I (the candidate) must review and be able to explain every module.

Issues found in AI output / changes after review
* The first file-creation command used a shell shortcut that did not create the
  sub-folders; it was noticed and the files were re-created.
* A design choice to review: category is collected from the 50 category pages instead
  of 1,000 book pages, to keep the run short (see README).
* <Add your own findings here after you run and read the code. Example: anything you
  changed, any selector you had to fix on the live site.>

How it was verified
* 31 offline unit tests (cleaning, validation, deduplication, scrapers with fake HTML,
  retry logic against a local test server, full pipeline).
* An end-to-end run of `main.py` against a local mock server.
* The Books to Scrape home page structure was checked against the live site
  (1,000 results, 50 pages, 50 categories, next link, title attribute, price format).
* NOT done by the AI: a complete live run against both websites (no internet access
  in the build environment). Quote-site selectors come from the standard layout of
  that site.
* <Add after YOUR live run: rows in the CSV, the summary_report.json totals, and
  a few records you compared by hand with the website.>